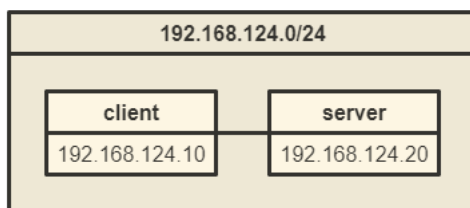


Network Security Q4 Y2024-2025

Assignment: Network Layer

IP Spoofing (20 points)

In this assignment, you will perform an IP spoofing attack. Consider the setup below:



The server hosts a secret on port 80, but this port is secured by a ICMP port knocking mechanism. It keeps port 80 closed until two ICMP ping packets with a payload length of 22 are sent to the server. ICMP is only accepted from the client's IP address (192.168.124.10). Port 80 accepts connections from any IP after it is opened.

The client can send two ICMP pings as such:

```
ping -c 2 -s 22 192.168.124.20
```

Environment

The provided environment contains three hosts: the client container (192.168.124.10), the server container (192.168.124.20), and an attacker container (192.168.124.30) which you will be working from. Set up the environment as such:

```
docker compose build
docker compose up -d
```

If you get an error, you probably didn't tear down the environment of the previous assignment: go back to its directory and run `docker compose down`.

The *attacker* container has a mount configured, such that you can edit files directly on your computer in an editor of your choice and then access them within the container: files in the `solution` directory are mirrored in the container under `/solution`. You can enter the container by executing the following command.

```
docker exec -ti assignment_3_spoofing_attacker bash
```

After finishing the assignment, make sure to clean up the environment using the command below.

```
docker compose down
```

Goal

Craft a Python3 script to target this ICMP port knocking mechanism from the *attacker* machine, spoofing the client's IP address in the process, retrieve the secret from the HTTP server running on port 80, and print it to the terminal. The script should accept the IP address we are targeting, the IP address we are spoofing the request from and the payload length as arguments:

```
python3 icmp_spoof.py 192.168.124.20 192.168.124.10 22
```

Craft the packets yourself from within the script using Scapy. For retrieving the secret, you may use the provided **requests** Python library. The assignment is considered successful if the script successfully retrieves the secret from the server. For debugging, you can also use **curl** to check if the port has been opened:

```
curl http://192.168.124.20
```

Hint: Once the port knock succeeds, you can close the port again by restarting the server:

```
docker compose restart server
```

You may not use external libraries or tools different from those installed on the provided *attacker* container.

Submission Instructions

After completing the assignment and verifying that it works correctly, submit your Python script on WebLab.